

DOL-Theme

Vorlage für Klausuraufgaben der Deutschen Linguistikolympiade (DOL)

Version 0.1.0

September 2026

1. Initialisierung

Um das DOL-Theme zu verwenden, importiere es zunächst und setze anschließend die Konfiguration über die `#show`-Regel `dol-theme`.

```
#import "@preview/dol-theme:0.1.0": *
#show: dol-theme.with(
  title: [Rätselsprache],
  author: [Jonta Ikaluk],
  points: 20,
  composition: [8+3+9]
)
```

2. Rätsel

Jedes Rätsel wird in einer eigenen `typst`-Datei gesetzt und mit obiger Initialisierung begonnen.

Jede Teilaufgabe wird durch `#task` eingeleitet. Die Punkte der Teilaufgaben werden als Summe `composition` in der Initialisierung angegeben.

```
#task Ordne die deutschen Übersetzungen den Sätzen der Rätselsprache zu.
```

3. Counter

Das DOL-Theme bietet zwei Funktionen zur Inkrementierung und Darstellung von Countern.

```
#let a = counter("raetsel-a")

#numstep(a) lipu
#numstep(a) ni
#numstep(a) li
#a.update(0)
#letstep(a) nasin
#letstep(a) kepeken
#letstep(a) sitelen
```



1. lipu
2. ni
3. li

- A. nasin
- B. kepeken
- C. sitelen

4. Tabellen

Folgende Tabellen-Designs sind für das Darstellen der Rätsel-Daten voreingestellt. Ein Tabellen-Designs wird mit `#show: ... -table` für alle nachfolgenden Tabellen aktiviert. Um ein Tabellen-Design nur für eine Tabelle anzuwenden, platziere entsprechende `#show`-Regel und diese Tabelle in eine Klammer, um einen lokalen Skopus zu erstellen.

4.1. matching-table

Bei Aktivierung des DOL-Themes ist dies das voreingestellte Tabellenvormat.

Diese Tabelle ist für solche Aufgaben, bei denen Phrasen der Rätselsprache den deutschen Entsprechungen zugeordnet werden sollen.

Hierbei wird die Rätselsprache fett, die deutschen (unsortierten) Übersetzungen kursiv linksbündig ausgedruckt. Alle Counter (Spalten 0 und 2) sind rechtsbündig.

```
#let a = counter("tokipona")
#let b = counter("tokipona2")
#show: matching-table
#table(
  numstep(a), [mi jan pi nasin sewi], letstep(b), [Sie ist keine Angestellte.],
  numstep(a), [ona li jan pali ala], letstep(b), [Er befindet sich hier im Haus.],
  numstep(a), [sina jan pi kama sona anu seme], letstep(b), [Bist du ein Student?],
  numstep(a), [ona li lon tomo ni], letstep(b), [Ich bin ein Mönch.],
)
```

- | | |
|--|--|
| 1. mi jan pi nasin sewi | A. <i>Sie ist keine Angestellte.</i> |
| 2. ona li jan pali ala | B. <i>Er befindet sich hier im Haus.</i> |
| 3. sina jan pi kama sona anu seme | C. <i>Bist du ein Student?</i> |
| 4. ona li lon tomo ni | D. <i>Ich bin ein Mönch.</i> |

4.2. translation-table

Diese Tabelle ist für Phrasen der Rätselsprache mitsamt den zugeordneten deutschen Übersetzungen. Die Einstellungen sind wie die der matching-table, nur ohne den zweiten Counter.

```
#let a = counter("tokipona3")
#show: translation-table
#table(
  numstep(a), [soweli li lon tomo lipu lili], [Die Katze befindet sich im Karton.],
  numstep(a), [ni li tomo mi ala], [Das ist nicht mein Haus.],
  numstep(a), [ona li jan pi kama sona], [Er ist Student.],
)
```

- | | |
|--|---|
| 1. soweli li lon tomo lipu lili | <i>Die Katze befindet sich im Karton.</i> |
| 2. ni li tomo mi ala | <i>Das ist nicht mein Haus.</i> |
| 3. ona li jan pi kama sona | <i>Er ist Student.</i> |

4.3. number-table

Diese Tabelle verzichtet auf beide Counter und richtet die in der rechten Spalte stehenden Ziffern rechtsbündig aus.

```
#show: number-table
#table(
  [wan], [1],
  [tu wan], [3],
  [ale mute mute mute luka tu wan], [168],
  [kijetesantakalu kijetesantakalu], [$x: 6 <= x <= 36$]
)
```

wan	1
tu wan	3
ale mute mute mute luka tu wan	168
kijetesantakalu kijetesantakalu	$x : 6 \leq x \leq 36$

4.4. booktabs-table

Diese Tabelle formatiert horizontale Linien entsprechend des *booktabs*-Stils zur Strukturierung von Regeln.

```
#show: booktabs-table
#figure(
  table(
    columns: 3,
    [Person], [Pronomen], [Verbpartikel],
    [1], [mi], table.cell(rowspan: 2, align: horizon)[$emptyset$],
    [2], [sina],
    [3], [ona], [li]
  )
)
```

Person	Pronomen	Verbpartikel
1	mi	∅
2	sina	
3	ona	li

4.5. standard-table

Diese Tabelle setzt alle Parameter zurück, sodass individuelle Tabellen von Grund auf gestaltet werden können.

```
#show: standard-table
...
```

5. Lösungen

Die Lösungen eines Rätsels werden korrespondierend zu den Teilaufgaben des Rätsels in eine eigene Datei (z. B. **raetsel-solution.typ**) gesetzt. Verwende hierbei dieselbe **Initialisierung** wie bei Rätseln, füge aber den Parameter `solution=true` hinzu. Dadurch wird die Lösung als solche formatiert.

(Der Parameter `author` kann zwar angegeben werden, wird bei Lösungen allerdings nicht angezeigt.)

```
#import "@preview/dol-theme:0.1.0": *
#show: dol-theme.with(
  title: [Rätselsprache],
  points: 20,
  composition: [8+3+9],
  solution: true
)
```

Verlinke die `raetsel-solution.typ` in den Initialisierung der entsprechenden Aufgabendatei **raetsel.typ**. Das wird allerdings erst bei der **Veröffentlichung** verwendet.

```
#import "@preview/dol-theme:0.1.0": *
#show: dol-theme.with(
  title: [Rätselsprache],
  ...,
  solution-file: include "raetsel-solution.typ"
)
```

6. Rätsel-Set

Ein Rätsel-Set wird in einer eigenen typst-Datei (z. B. **dol26-1.typ**) aus mehreren Rätseln (im selben Ordner) über die `#show`-Regel `compile-problems` kompiliert.

Setze mit den Variablen `time` und `editors` die für die Runde angesetzte Bearbeitungszeit (in Worten) und alle Namen der Redaktion. Diese werden auf der ersten Seite des Rätsel-Sets eingefügt.

```
#import "@preview/dol-theme:0.1.0": compile-problems
#show: compile-problems.with(
  time: [3 Stunden und 30 Minuten],
  editors: [Person 1, Person 2, Person 3, ...]
)

#include "problem1.typ"
#include "problem2.typ"
#include "problem3.typ"

#counter(heading).update(0)

#include "problem1-solution.typ"
#include "problem2-solution.typ"
#include "problem3-solution.typ"
```

6.1. Jahr und Runde

Um das Rätselset zu erstellen, müssen jetzt noch die **globalen Variablen** `year` und `round` gesetzt werden. Das geht auf folgende drei Arten.

6.1.1. Manuelle Kompilierung

Bei manueller Kompilierung durch die Kommandozeile können die globalen Variablen durch `--input`-Parameter gesetzt werden.

```
typst compile --input year=2026 --input round=1 dol26-1.typ dol26-1.pdf
```

6.1.2. Tinymist & VSCode

Bei automatischer Kompilierung durch Tinymist in VSCode können die Input-Parameter durch eine JSON-Datei für den ganzen Ordner gesetzt werden. Dazu muss im entsprechenden Ordner ein Ordner `.vscode/` erstellt werden und darin die Datei `settings.json`:

```
{
  "tinymist.typstExtraArgs": [
    "--input", "year=2026",
    "--input", "round=1",
    "dol26-1.typ"
  ]
}
```

6.1.3. Rätselspezifisch

Die Parameter `year` und `round` können auch lokal in der **Initialisierung** der einzelnen Rätsel gesetzt werden. Eine globale Setzung der Variablen durch Input-Parameter überschreibt diese lokale Setzung.

```
#import "@preview/dol-theme:0.1.0": *
#show: dol-theme.with(
  ...,
  year: 2026,
  round: 1
)
```

6.2. Anonymisierung der Rätsel

Für Online-Runden können Rätsel durch setzen des Parameters `metadata=false` anonymisiert werden. Dabei werden der Autorenname und der Räseltitel versteckt.

Dieser Parameter kann sowohl durch **globale Variablen** als auch in der **Initialisierung** gesetzt werden. Er hat keinen Effekt, wenn der Parameter `publish==true`.

```
... --input metadata=false ...
```

```
..., "--input", "metadata=false", ...
```

```
#show: dol-theme.with(
  ...,
  metadata: false
)
```

7. Veröffentlichung

Für die spätere Veröffentlichung erhalten die Rätsel ein erweitertes Layout durch die globale Variable `publish=true`, die wie metadata `global` oder `lokal` gesetzt werden kann.

In der Initialisierung der Rätseldatei `raetsel.typ` kommen weitere Parameter hinzu. Diese werden nur im publish-Modus verwendet, können daher beim Setzen des Rätsels für die Klausur direkt mit angegeben werden.

```
#import "@preview/dol-theme:0.1.0": *
#show: dol-theme.with(
  title: [Rätselsprache],
  ...,
  longtitle: [Grammatisches Phänomen in der Rätselsprache],
  background: [Hintergrund und Quellen ...],
  solution-file: include "raetsel-solution.typ" // <- publish=true
)
```

- `longtitle`: Ein detaillierterer Titel, der im publish-Modus ausschließlich verwendet wird (auch für die Lizenz). Wenn nichts angegeben ist, wird `title` verwendet.
- `background`: Ein ganzer Paragraph mit Hintergrundinformationen, der den eigentlichen Aufgaben vorangestellt wird. Hier sollen auch Quellen zitiert werden.
- `solution-file`: Hier wird die entsprechende Lösungsdatei `raetsel-solution.typ` verlinkt, um sie in der gleichen Datei mit auszudrucken. Wichtig ist, dass `raetsel-solution.typ` ebenfalls `publish=true` eingestellt hat (global passiert das automatisch), und dass der Befehl `include` dem Dateinamen vorangestellt ist.

8. Konfigurationen

Vollständige Liste der Argumente von `dol-theme` und `compile-problems`. Die Defaultwerte sind bei Booleans die angegebenen, überall sonst `none`.

Außerdem die internen Konfigurationen der exportierten Funktionen.

8.1. Initialisierung (dol-theme)

```
#import "@preview/dol-theme:0.1.0": *
#show: dol-theme.with(
  title: [Rätselsprache],                                // hidden when metadata==false
  longtitle: [Grammatisches Phänomen in der Rätselsprache], // for publish
  author: [Jonta Ikaluk],                                // hidden when metadata==false
  points: 20,                                             // int
  composition: [8+3+9],                                // sum should equal points
  background: [Hintergrund und Quellen ...],             // for publish
  solution: false,                                       // DEFAULT      true in raetsel-solution.typ
  solution-file: include "raetsel-solution.typ", // for publish, only in raetsel.typ
  year: 2026,                                           // GLOBAL          ### These global parameters ###
  round: 1,                                           // GLOBAL          ### should be entered via ###
  metadata: true,                                     // GLOBAL DEFAULT  ### --input options during ###
  publish: false                                       // GLOBAL DEFAULT  ### compilation (see 6.1-7) ###
)
```

8.2. Rätsel-Set (compile-problems)

```
#import "@preview/dol-theme:0.1.0": compile-problems
#show: compile-problems.with(
  time: [3 Stunden und 30 Minuten],
  editors: [Person 1, Person 2, Person 3, ...],
  year: 2026,                      // GLOBAL          ### see ###
  round: 1,                        // GLOBAL          ### above ###
)
```

8.3. Counter

```
#let task-counter = counter("task")
#let task = {
  task-counter.step()
  box(width: 2em, context strong(task-counter.display("(a)")))
}
```

```
#let numstep(counter) = {
  counter.step()
  context counter.display("1.")
}
```

```
#let letstep(counter) = {
  counter.step()
  context counter.display("A.")
}
```

8.4. Tabellen

8.4.1. matching-table

```
#show table.cell.where(x: 1): set text(weight: "bold")
#show table.cell.where(x: 3): set text(style: "italic")
#set table(
  stroke: none,
  columns: 4,
  align: (right, left, right, left),
  column-gutter: (0em, 6em, 0em),
)
```

8.4.2. translation-table

```
#show table.cell.where(x: 1): set text(weight: "bold")
#show table.cell.where(x: 2): set text(style: "italic")
#set table(
  stroke: none,
  columns: 3,
  align: (right, left, left),
  column-gutter: (0em, 2em),
)
```

8.4.3. number-table

```
#show table.cell.where(x: 0): set text(weight: "bold")
#show table.cell.where(x: 1): set text(weight: "regular")
#set table(
  stroke: none,
  columns: 2,
  align: (left, right),
  column-gutter: 2em,
)
```

8.4.4. booktabs-table

```
#show table.cell: set text(weight: "regular")
#show table.cell: set text(style: "normal")
#show table: set table(
  stroke: (x, y) => (
    top: if y == 0 { 1pt } else if y == 1 { 0.7pt } else { 0pt },
    bottom: 1pt,
  ),
)
#set table(
  stroke: 1pt + black,
  columns: (),
  align: auto,
  column-gutter: (),
)
```

8.4.5. standard-table

```
#show table.cell: set text(weight: "regular")
#show table.cell: set text(style: "normal")
#set table(
  stroke: 1pt + black,
  columns: (),
  align: auto,
  column-gutter: (),
)
```